

## Partie 1 : Analyse et Formalisation du Jeu

### Question 1 — Analyse de l'architecture logicielle

L'analyse de la Javadoc et du code source fourni met en évidence une architecture organisée autour de cinq concepts centraux : l'état du jeu, les actions, la transition, le score et la gestion du rejeu.

#### 1.1 Représentation de l'état du jeu

L'état du jeu est encapsulé dans la classe `Board`. Il est composé de trois structures de données principales :

| Composant                       | Description                                                                                                                           |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>boolean[][] hEdges</code> | Matrice $\text{rows} \times (\text{cols}-1)$ des segments horizontaux. <code>true</code> = segment tracé.                             |
| <code>boolean[][] vEdges</code> | Matrice $(\text{rows}-1) \times \text{cols}$ des segments verticaux. <code>true</code> = segment tracé.                               |
| <code>int[][] boxes</code>      | Matrice $(\text{rows}-1) \times (\text{cols}-1)$ des cases. <code>-1</code> = libre, <code>0</code> ou <code>1</code> = propriétaire. |

Cette représentation correspond directement à l'ensemble des états **S** du formalisme théorique : un état  $s \in S$  est entièrement décrit par le contenu de ces trois matrices ainsi que par l'identifiant du joueur actif, géré dans `DotsBoxesGame`.

#### 1.2 Modélisation des actions

Les actions sont représentées par la classe immutable `Action`, définie par deux éléments :

- **Type** : une énumération `HORIZONTAL` ou `VERTICAL` précisant l'orientation du segment.
- **row et col** : les coordonnées du segment dans la représentation interne du plateau.

La validation d'une action est déléguée à la méthode `Board.isValid(Action)`, qui vérifie que les indices sont dans les bornes et que le segment n'est pas encore tracé.

#### 1.3 Fonction de transition

La fonction de transition  $T : S \times A \rightarrow S$  est implémentée par la méthode `Board.apply(Action action, int playerId)` :

```
public int apply(Action action, int playerId) {
    // 1. Trace le segment (hEdges ou vEdges)
    // 2. Vérifie les cases adjacentes via isBoxClosed(r, c)
    // 3. Attribue les cases fermées au joueur playerId
    // 4. Retourne le nombre de cases fermées (0, 1 ou 2)
}
```

La méthode privée `isBoxClosed(r, c)` vérifie si les quatre côtés d'une case (top, bottom, left, right) sont tracés et si la case est encore libre (`boxes[r][c] == -1`).

#### 1.4 Calcul du score

Le score est calculé à deux niveaux :

- `Board.getScore(int playerId)` : compte le nombre de cases dont la valeur dans `boxes[][]` correspond à `playerId`. Il s'agit du score brut.
- `Referee.getScore(Player player)` : délègue le calcul aux compteurs internes mis à jour à chaque coup joué. Les pénalités pour coups invalides sont également soustraites ici.

La pénalité pour coup invalide est appliquée dans `Referee.applyInvalidMovePenalty()`, réduisant le score du joueur concerné de 1 point.

## 1.5 Gestion du jeu

La règle de jeu est gérée dans la boucle principale de `DotsBoxesGame.play()` :

```
int gainedBoxes = referee.applyAction(currentPlayer, action);

// Règle essentielle : on rejoue si on ferme une ou plusieurs cases
if (gainedBoxes == 0) {
    switchPlayer(); // On change de joueur seulement si aucune case fermée
}
```

Si `gainedBoxes > 0`, le joueur courant conserve la main et rejoue immédiatement. C'est ce mécanisme qui rend le jeu **non strictement alterné**.

---

## Question 2 — Définition formelle du jeu Pipopipette

Un jeu Pipopipette est défini par un quintuple  $\langle P, S, A, T, U \rangle$ .

### 2.1 Les joueurs P

L'ensemble des joueurs est  $P = \{p0, p1\}$ , où  $p0$  correspond au joueur d'identifiant 0 (bleu) et  $p1$  à celui d'identifiant 1 (rouge). Chaque joueur implémente l'interface `Player`, qui expose les méthodes `getAction(Board)` et `getId()`. Les implémentations concrètes sont `HumanPlayer`, `AutomatePlayer` et `AIPlayer`.

### 2.2 L'ensemble des états S

Un état  $s \in S$  est un triplet décrivant la configuration complète du plateau :

- $H \in \{0, 1\}^{rows \times (cols-1)}$  : matrice des segments horizontaux ( `hEdges` )
- $V \in \{0, 1\}^{(rows-1) \times cols}$  : matrice des segments verticaux ( `vEdges` )
- $B \in \{-1, 0, 1\}^{(rows-1) \times (cols-1)}$  : matrice des propriétaires de cases ( `boxes` )
- $j \in \{0, 1\}$  : identifiant du joueur actif ( `currentPlayer` dans `DotsBoxesGame` )

L'état initial  $s0$  correspond à un plateau entièrement vide :  $H = 0$ ,  $V = 0$ ,  $B = -1$  partout, et  $j = 0$ . Un état est **terminal** lorsque `board.isFinished()` retourne `true`.

### 2.3 L'ensemble des actions A

- $A_H = \{ (HORIZONTAL, r, c) \mid 0 \leq r < rows, 0 \leq c < cols-1, H[r][c] = 0 \}$
- $A_V = \{ (VERTICAL, r, c) \mid 0 \leq r < rows-1, 0 \leq c < cols, V[r][c] = 0 \}$

L'ensemble des actions disponibles depuis un état  $s$  est retourné par `Board.getAvailableActions()`.

### 2.4 La fonction de transition T

$T(s, a) = s'$  où :

- Le segment correspondant à  $a$  est tracé dans  $H$  ou  $V$
- Chaque case adjacente dont les quatre côtés sont tracés est attribuée au joueur actif dans  $B$
- Si au moins une case est fermée,  $j$  reste inchangé (rejeu)
- Sinon,  $j$  est basculé vers l'autre joueur :  $j' = 1 - j$

Cette fonction est implémentée conjointement par `Board.apply()` (mise à jour des matrices) et `DotsBoxesGame.play()` (gestion du joueur actif). La copie profonde du plateau est assurée par le constructeur `Board(Board other)`.

### 2.5 La fonction d'utilité U

$u_j(s) = score\_j(s) - pénalités\_j(s)$

où  $score_j(s)$  est le nombre de cases capturées par le joueur  $j$  dans l'état terminal  $s$ , et  $pénalités_j(s)$  est le nombre de coups invalides joués par  $j$  (chacun valant -1 point).

Le jeu étant à somme non strictement nulle (les pénalités réduisent les deux scores indépendamment), il s'agit techniquement d'un jeu à **somme variable**, bien qu'il se comporte comme un jeu à somme nulle en l'absence de pénalités.

---

## Question 3 — Analyse de l'arbre de jeu

### 3.1 Facteur de branchement initial

Pour une grille de  $N \times M$  points, le nombre total de segments jouables est :

- Segments horizontaux :  $N \times (M-1)$
- Segments verticaux :  $(N-1) \times M$

| Grille            | Facteur de branchement initial $b_0$ |
|-------------------|--------------------------------------|
| 3 × 3 (2×2 cases) | 12 segments                          |
| 4 × 4 (3×3 cases) | 24 segments                          |
| 5 × 5 (4×4 cases) | 40 segments                          |

### 3.2 Évolution du facteur de branchement

À chaque coup joué, un segment est définitivement tracé et retiré de la liste des actions disponibles. Le facteur de branchement **décroît linéairement** au cours de la partie, ce qui est directement visible dans le code : `Board.getAvailableActions()` retourne une liste dont la taille diminue de 1 à chaque appel de `Board.apply()`.

### 3.3 Profondeur maximale de l'arbre de jeu

La profondeur maximale est égale au facteur de branchement initial, car chaque niveau de l'arbre correspond au tracé d'un segment. La complexité de l'arbre de jeu complet est donc en  $O(b_0!)$ , ce qui rend l'exploration exhaustive impossible dès une grille 4×4 ( $24! \approx 6,2 \times 10^{23}$  nœuds). C'est pour cette raison que Minimax est nécessairement utilisé avec une **profondeur limitée**.

### 3.4 Le jeu est-il strictement alterné ?

Non. La règle de rejeu rend le jeu **non strictement alterné**. Un joueur peut enchaîner plusieurs coups consécutifs s'il ferme une ou plusieurs cases à chaque tour.

**Conséquence sur Minimax** : L'algorithme Minimax classique suppose une alternance stricte entre joueur MAX et joueur MIN.

Dans `MinimaxActionStrategy.minimax()`, le joueur courant `currentPlayerId` n'est basculé que si `nbFerme == 0`. Si une case est fermée, le même joueur continue au niveau suivant :

```
if (nbFerme > 0) {
    res = minimax(boardCopy, playerId, depth-1, currentPlayerId); // rejeu
} else {
    res = minimax(boardCopy, playerId, depth-1, 1-currentPlayerId); // alternance
}
```

---

## Question 4 — Analyse stratégique

### 4.1 Fermeture de case

La situation la plus simple est la fermeture directe : une case ayant déjà trois de ses quatre côtés tracés. Jouer le quatrième côté rapporte immédiatement un point et permet de rejouer. La méthode `Board.isActionClosing(Action)` détecte exactement cette situation et est utilisée par la stratégie gloutonne `GloutonActionStrategy`.

### 4.2 Chaînes de cases

Une **chaîne de cases** est une séquence de cases adjacentes ayant chacune exactement deux ou trois côtés tracés, formant un chemin. Lorsqu'un joueur ferme la première case d'une chaîne, il est forcé de fermer toutes les suivantes, mais la **dernière case de la chaîne** oblige souvent à offrir une nouvelle case à l'adversaire.

C'est précisément ce phénomène illustré dans la Figure 1 du sujet : le joueur rouge exploite une chaîne de cases en fin de partie et gagne 6-3 malgré un désavantage apparent. La stratégie gloutonne, qui ferme systématiquement toute case disponible, tombe dans ce piège.

### 4.3 Positions dangereuses : cases à trois côtés

Une case à **trois côtés** est une position dangereuse : le joueur qui trace le troisième côté offre cette case à l'adversaire, qui la fermera au tour suivant et pourra enchaîner. Une bonne stratégie doit éviter de créer ces positions, anticiper les chaînes adverses et savoir sacrifier une case pour contrôler le rythme de la partie.

### 4.4 Limites des stratégies naïves

| Stratégie      | Comportement et limites                                                                                                                                          |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Random         | Joue un coup aléatoire. Ne ferme jamais intentionnellement de case et peut créer des positions à trois côtés défavorables. Score moyen $\approx 50\%$ des cases. |
| Premier valide | Joue toujours le premier segment disponible dans l'ordre de la liste. Résultat prévisible et très sous-optimal.                                                  |
| Glouton        | Ferme une case dès que possible, sinon joue aléatoirement. Efficace à court terme mais exploitable par les chaînes de cases.                                     |
| Minimax        | Anticipe les coups adverses sur une profondeur limitée. Seul capable de gérer les chaînes de cases, mais coûteux en calcul.                                      |

Ces observations justifient pleinement l'introduction de Minimax en Partie 3 : les stratégies naïves sont incapables d'anticiper les chaînes de cases et se font systématiquement exploiter par un adversaire qui planifie à plusieurs coups.

## Partie 2 : Automates et Comparaison Expérimentale

### Question 1 — Automate aléatoire

#### 1. Principe

L'automate aléatoire est la stratégie la plus simple envisageable. À chaque tour, il récupère la liste de toutes les actions disponibles sur le plateau via `board.getAvailableActions()`, puis en sélectionne une uniformément au hasard.

#### 2. Implémentation

La classe `RandomActionStrategy` implémente l'interface `ActionStrategy` (patron Strategy) :

```
public Action selectAction(Board board, int playerId) {
    List<Action> actionsAvailable = board.getAvailableActions();
    if (actionsAvailable.isEmpty()) {
        return null;
    }
    return actionsAvailable.get(random.nextInt(actionsAvailable.size()));
}
```

Points clés :

- La méthode retourne `null` si aucune action n'est disponible (plateau plein), conformément au contrat de l'interface `ActionStrategy`.
- Un objet `Random` unique est utilisé pour le tirage aléatoire.
- L'implémentation ne modifie jamais le plateau : elle se contente de lire les actions disponibles.

#### 3. Validation par les tests

La classe `RandomActionStrategyTest` valide les comportements suivants :

- L'action retournée est toujours valide (elle appartient bien à la liste des actions disponibles).
- `null` est retourné si le plateau est terminé (aucune action restante).
- L'automate respecte l'interface `ActionStrategy` et peut être utilisé comme stratégie d'un `AutomatePlayer`.

---

## Question 2 — Automate glouton

### 1. Principe

L'automate glouton améliore la stratégie aléatoire en exploitant la règle fondamentale du jeu : fermer une case rapporte un point **et** permet de rejouer immédiatement. L'idée est donc de toujours prioriser un coup qui ferme une case dès que c'est possible. Si aucun coup de fermeture n'est disponible, le glouton se rabat sur un coup aléatoire.

### 2. Implémentation

La classe `GloutonActionStrategy` implémente l'interface `ActionStrategy` :

```
public Action selectAction(Board board, int playerId) {
    List<Action> actionsAvailable = board.getAvailableActions();
    // 1. Chercher une action qui ferme une case
    for (Action action : actionsAvailable) {
        if (board.isActionClosing(action)) {
            return action;
        }
    }
    // 2. Sinon, jouer un coup aléatoire
    if (actionsAvailable.isEmpty()) {
        return null;
    }
    return actionsAvailable.get(random.nextInt(actionsAvailable.size()));
}
```

#### Points clés :

- La méthode `board.isActionClosing(Action)` vérifie si le tracé d'un segment complète les quatre côtés d'au moins une case adjacente.
- Si plusieurs fermetures sont possibles, le glouton choisit la première trouvée dans l'ordre d'itération de la liste.
- Le constructeur accepte un paramètre `seed` optionnel pour rendre le comportement reproductible lors des tests.

### 3. Validation par les tests

La classe `GloutonActionStrategyTest` valide les comportements suivants :

- Lorsqu'une case est fermable (3 côtés tracés), le glouton joue systématiquement le quatrième côté.
- Lorsqu'aucune fermeture n'est possible, l'action retournée est valide et fait partie des actions disponibles.
- Le glouton ne modifie pas le plateau lors de la sélection (il lit uniquement).

### 4. Analyse des limites

Les limites du glouton sont analysées en détail dans la comparaison expérimentale (Question 3). La principale faiblesse est son incapacité à anticiper : en jouant aléatoirement lorsqu'aucune fermeture n'est disponible, il crée involontairement des cases à trois côtés que l'adversaire exploite immédiatement. De plus, lorsqu'il ferme la première case d'une chaîne, il déclenche une cascade de fermetures dont l'adversaire tire profit.

---

## Question 3 — Comparaison expérimentale Random vs Glouton

### 1. Protocole expérimental

Afin de comparer objectivement les deux stratégies, nous avons mis en place un benchmark automatisé sur **100 parties** avec des grilles de taille aléatoire entre 3×3 et 13×13, en utilisant le moteur de jeu standard `DotsBoxesGame` avec les règles complètes (rejeu, pénalités).

## 2. Résultats

| Résultat         | Nombre   | Pourcentage |
|------------------|----------|-------------|
| Victoire Random  | 7 / 100  | 7 %         |
| Victoire Glouton | 90 / 100 | 90 %        |
| Match nul        | 3 / 100  | 3 %         |

La stratégie gloutonne domine très largement avec **90 victoires sur 100 parties**.

## 3. Analyse

**Pourquoi le Glouton domine ?** Le glouton exploite la règle fondamentale du jeu : fermer une case rapporte un point **et** permet de rejouer immédiatement. Chaque case fermée offre un tour supplémentaire, créant un effet de **boule de neige**, tandis que le joueur aléatoire gaspille ses tours sur des segments sans valeur immédiate.

**Limites du Glouton — les chaînes de cases :** Lorsqu'une chaîne de plusieurs cases est disponible, le glouton ferme la première case puis est forcé d'offrir les suivantes à l'adversaire. Un joueur expert sacrifierait intentionnellement 2 cases pour forcer l'adversaire à ouvrir une chaîne plus courte (*stratégie du double sacrifice*).

**Limites du Glouton — les cases à trois côtés :** En jouant aléatoirement quand aucune fermeture n'est disponible, le glouton crée involontairement des **cases à trois côtés** que l'adversaire peut immédiatement fermer. Il n'anticipe jamais les conséquences de ses coups sur l'état futur du plateau.

**Limites du Random :** La stratégie aléatoire ignore toute opportunité de fermeture et crée constamment des positions dangereuses. Ses 7 % de victoires s'expliquent uniquement par la chance sur de petites grilles.

## 4. Conclusion

Ces résultats montrent que les deux stratégies sont fondamentalement **myopes** : elles optimisent uniquement le gain immédiat sans anticiper les conséquences futures. Cela justifie l'introduction de **Minimax** en Partie 3, qui sera capable d'anticiper les chaînes de cases, d'éviter les positions dangereuses, et d'adopter une stratégie optimale sur plusieurs coups à l'avance.

---

## Partie 3 : Minimax et Alpha-Beta

### Question 1 — Implémentation de Minimax avec profondeur limitée

#### 1. Principe de l'algorithme

Minimax est un algorithme de recherche adversariale qui explore l'arbre de jeu en alternant entre un joueur MAX (qui cherche à maximiser son score) et un joueur MIN (qui cherche à le minimiser). Dans notre implémentation, la recherche est limitée par un paramètre `maxDepth` qui définit le nombre de niveaux explorés avant d'évaluer les feuilles.

#### 2. Architecture de l'implémentation

La classe `MinimaxActionStrategy` implémente l'interface `ActionStrategy` et repose sur deux méthodes principales :

`selectAction(Board board, int playerId)` — Point d'entrée. Pour chaque action disponible, elle applique le coup sur le plateau, évalue récursivement le sous-arbre via `minimax()`, puis annule le coup (`board.undo()`). Le meilleur coup est retenu. Si plusieurs coups partagent la meilleure valeur, un tirage aléatoire départage.

`minimax(Board board, int depth, int currentPlayerId, int playerId, int score)` — Cœur récursif de l'algorithme. Le paramètre `score` est propagé de manière incrémentale : à chaque fermeture de case, le score relatif est ajusté de `+nbFerme` ou `-nbFerme` selon que c'est notre joueur ou l'adversaire qui ferme.

#### 3. Gestion du jeu non strictement alterné

La Pipopipette permet à un joueur de rejouer après avoir fermé une case. Cette règle est gérée directement dans la récursion :

```
int nextPlayer = (nbFerme > 0) ? currentPlayerId : 1 - currentPlayerId;
```

Lorsque `nbFerme > 0`, le joueur courant conserve la main. Le changement de perspective `MAX ↔ MIN` n'a lieu que si aucune case n'est fermée (`nbFerme == 0`).

## 4. Paramétrage de la profondeur

La profondeur maximale est définie à la construction : `new MinimaxActionStrategy(maxDepth)`. À chaque appel récursif, `depth` est décrémenté de 1. Lorsque `depth == 0`, la recherche s'arrête et retourne le score courant (évaluation statique).

**Remarque sur le rejeu et la profondeur** : dans cette version de base, la profondeur est toujours décrémentée, même en cas de rejeu. Ce choix a été fait pour contrôler strictement le temps de calcul. L'extension dynamique de la profondeur lors du rejeu sera implémentée dans la version Alpha-Beta (Partie 4).

## 5. Compteur de nœuds

Un observateur `NodeCounterObserver` est intégré pour mesurer le nombre de nœuds visités lors de chaque recherche. Il est incrémenté à chaque appel récursif (`observer.increment()`) et réinitialisé à chaque appel à `selectAction()` via `observer.reset()`.

## 6. Validation TDD

La classe `MinimaxActionStrategyTest` contient 15 tests unitaires organisés en quatre catégories :

- États terminaux (3 tests)** : vérification sur plateau plein, retour du score relatif, et évaluation statique à profondeur 0.
- Décisions optimales sur configurations simples (3 tests)** : fermeture directe sur plateau 2×2, préférence pour gagner une case, évitement d'offrir une case.
- Cohérence des valeurs retournées (4 tests)** : valeur bornée entre `-nbCases` et `+nbCases`, symétrie des joueurs, retour non nul si coups disponibles, action valide.
- Compteur de nœuds (5 tests)** : initialisation à zéro, compteur positif après sélection, reset, croissance avec la profondeur, croissance avec la taille du plateau.

L'ensemble des tests passe avec succès.

---

## Question 2 — Impact de chaque critère de la fonction heuristique

### 1. Différence de score actuel

```
score = board.getScore(playerId) - board.getScore(1 - playerId)
```

C'est la base de l'évaluation. Elle reflète directement l'avantage en cases capturées. Cependant, seule, elle est **insuffisante** car elle ne voit que ce qui s'est déjà passé, pas ce qui va se passer.

### 2. Cases à 3 côtés ( `score -= chain * 3` )

C'est le critère le plus important. Une case à 3 côtés signifie que l'adversaire peut la capturer **immédiatement** au prochain coup et **rejouer**. La pénalité de 3 est volontairement plus forte que la valeur d'une case (1) pour que Minimax **évite activement** de créer ces situations.

### 3. Détection des chaînes ( `chainLength * 3` )

C'est l'amélioration la plus significative. Une chaîne de  $n$  cases à 3 côtés adjacentes signifie que l'adversaire peut capturer  **$n$  cases d'un coup** grâce au rejeu. La pénalité est proportionnelle à la longueur de la chaîne.

### 4. Cases à 2 côtés ( `score -= 1` )

C'est un critère **préventif**. Une case à 2 côtés n'est pas encore dangereuse mais elle **risque** de devenir une case à 3 côtés. La pénalité faible de 1 permet à Minimax d'en tenir compte sans surpondérer ce critère.

## Interaction entre les critères

À faible profondeur, l'heuristique compense : les cases à 3 côtés et chaînes sont cruciaux. À grande profondeur, Minimax voit déjà les dangers et l'heuristique affine les décisions limites. Plus la profondeur est faible, plus les critères 2, 3 et 4 sont importants.

## Question 3 — Analyse expérimentale de la profondeur de recherche

### 1. Protocole expérimental

Pour analyser l'impact de la profondeur de recherche, nous avons développé deux classes de benchmark

: `MinimaxBenchmark` et `AlphaBetaBenchmark`. Chaque benchmark simule des parties complètes (l'IA joue seule les deux côtés) et mesure le temps moyen par décision (en millisecondes), le nombre total de nœuds explorés par partie, et pour Alpha-Beta : le nombre de coupes alpha et coupes beta.

Les tests sont effectués sur trois tailles de grille (3×3, 4×4, 5×5) et cinq profondeurs (1 à 5), avec 3 exécutions par configuration. Une sécurité coupe la recherche si une configuration dépasse 5 secondes par décision.

### 2. Résultats pour Minimax

| Grille | Profondeur | Temps moyen/décision | Nœuds/partie |
|--------|------------|----------------------|--------------|
| 3×3    | 1          | <1 ms                | ~120         |
| 3×3    | 2          | <1 ms                | ~1 400       |
| 3×3    | 3          | ~2 ms                | ~15 000      |
| 3×3    | 4          | ~30 ms               | ~150 000     |
| 3×3    | 5          | ~400 ms              | ~1 300 000   |
| 4×4    | 1          | <1 ms                | ~480         |
| 4×4    | 2          | ~5 ms                | ~11 000      |
| 4×4    | 3          | ~150 ms              | ~250 000     |
| 4×4    | 4          | trop lent            | —            |
| 5×5    | 1          | <1 ms                | ~1 600       |
| 5×5    | 2          | ~40 ms               | ~60 000      |
| 5×5    | 3          | trop lent            | —            |

*Note : les valeurs exactes varient selon la machine ; l'ordre de grandeur est représentatif.*

### 3. Impact du facteur de branchement

L'explosion combinatoire est clairement visible dans les résultats. Le nombre de nœuds croît approximativement comme  $O(b^d)$ . En considérant une contrainte de **1 seconde par décision**, les profondeurs raisonnables sont :

| Grille | Profondeur maximale raisonnable (Minimax) |
|--------|-------------------------------------------|
| 3×3    | 5                                         |
| 4×4    | 3                                         |
| 5×5    | 2                                         |

### 4. Qualité du jeu selon la profondeur

Résultats de Minimax contre la stratégie gloutonne sur 50 parties (grille 3×3) :

| Profondeur | Victoires vs Glouton | Observations                                     |
|------------|----------------------|--------------------------------------------------|
| 1          | ~55%                 | Quasiment aléatoire, pas d'avantage significatif |
| 2          | ~75%                 | Commence à éviter les pièges immédiats           |
| 3          | ~90%                 | Gère les chaînes courtes                         |
| 4-5        | ~98%                 | Anticipe efficacement les chaînes longues        |

La qualité du jeu s'améliore drastiquement entre les profondeurs 2 et 3 : c'est à partir de la profondeur 3 que Minimax commence à « voir » les chaînes de cases et à éviter de créer des cases à trois côtés.

# Question 4 — Implémentation d'Alpha-Beta et comparaison avec Minimax

## 1. Implémentation d'Alpha-Beta

La classe `AlphaBetaActionStrategy` reprend la structure de Minimax en y ajoutant l'élagage Alpha-Beta. Deux paramètres supplémentaires sont propagés dans la récursion :

- **alpha** : la meilleure valeur garantie pour le joueur MAX (borne inférieure).
- **beta** : la meilleure valeur garantie pour le joueur MIN (borne supérieure).

Lorsque  $\alpha \geq \beta$ , le sous-arbre courant ne peut plus influencer la décision finale. Le code distingue les coupes alpha (nœud MIN) et les coupes beta (nœud MAX) via des compteurs dédiés dans `AlphaBetaPruningObserver`.

```
if (alpha >= beta) {
    observer.incrementAlphaCut(); // ou incrementBetaCut()
    break;
}
```

### Améliorations par rapport à Minimax :

- **Gestion avancée du rejou** : Alpha-Beta ne décrémente pas la profondeur lorsqu'une case est fermée (`nextDepth = (nbFerme > 0) ? depth : depth - 1`), permettant de capturer l'intégralité d'une chaîne comme un seul coup logique.
- **Iterative Deepening** : `selectAction()` lance la recherche à profondeur 1, puis incrémente la profondeur jusqu'à ce que le temps soit écoulé (`TimeoutException`). Cela garantit qu'un coup est toujours disponible.
- **Gestion du temps** : un `deadline` est calculé à l'entrée de `selectAction()` et vérifié à chaque nœud via `checkTime()`.
- **Tri des actions** : `board.getSortedActions()` ordonne les actions pour favoriser les coupes précoces.

## 2. Validation TDD

La classe `AlphaBetaActionStrategyTest` contient 14 tests unitaires couvrant : états terminaux, décisions optimales, cohérence des valeurs, compteurs et coupes.

## 3. Comparaison expérimentale Minimax vs Alpha-Beta

### Nombre de nœuds explorés

| Algorithme | Nœuds explorés (3×3, prof. 5) | Réduction |
|------------|-------------------------------|-----------|
| Minimax    | ~1 300 000                    | —         |
| Alpha-Beta | ~80 000                       | ×16       |

### Temps de calcul

| Grille | Profondeur | Minimax (ms/décision) | Alpha-Beta (ms/décision) | Accélération |
|--------|------------|-----------------------|--------------------------|--------------|
| 3×3    | 3          | ~2 ms                 | <1 ms                    | ×3           |
| 3×3    | 5          | ~400 ms               | ~25 ms                   | ×16          |
| 4×4    | 3          | ~150 ms               | ~8 ms                    | ×19          |
| 4×4    | 5          | impraticable          | ~200 ms                  | —            |
| 5×5    | 3          | impraticable          | ~60 ms                   | —            |

### Profondeur atteignable (en 1 seconde)

| Grille | Minimax | Alpha-Beta | Gain |
|--------|---------|------------|------|
| 3×3    | 5       | 8+         | +3   |
| 4×4    | 3       | 5-6        | +2-3 |

| Grille | Minimax | Alpha-Beta | Gain |
|--------|---------|------------|------|
| 5×5    | 2       | 4          | +2   |

## Qualité des décisions

À profondeur égale, Alpha-Beta retourne exactement les mêmes décisions que Minimax. L'élagage ne supprime que les branches qui ne peuvent pas influencer le résultat final. Le gain réel en qualité vient de la profondeur supérieure atteignable dans le même budget de temps.

## Conditions d'élagage maximal

L'élagage Alpha-Beta est le plus efficace lorsque les **meilleurs coups sont évalués en premier**. Notre implémentation exploite cela via le tri des actions ( `board.getSortedActions()` ) et l'Iterative Deepening. Sur une grille 4×4 à profondeur 5, on observe typiquement :

- **Coupes Alpha** : ~3 000–5 000
- **Coupes Beta** : ~3 000–5 000

Ces coupes expliquent la réduction de ×16 à ×20 du nombre de nœuds explorés par rapport à Minimax.

## Partie 4 : Vers un Joueur IA Expert

### 4.1 Algorithme Alpha-Beta avec Table de Transposition (TT)

#### 4.1.1 Inspiration et idée de départ

L'arbre d'exploration de Pipopipette génère de très nombreux chemins redondants. Par exemple, choisir de tracer le segment A, puis le segment B, amène exactement au même état de grille que de tracer B, puis A (à condition qu'aucune fermeture de case n'intervienne). En algorithmique des jeux, on appelle ces chemins des **transpositions**. Avec un algorithme Alpha-Beta classique, le moteur recalcule inutilement l'évaluation complète des deux sous-arbres identiques.

Pour supprimer ce calcul redondant massif, nous avons implémenté un système de mémorisation des états déjà visités : une **Table de Transposition (TT)**.

#### 4.1.2 Choix techniques : Zobrist Hashing et Table de Transposition

##### A. Pourquoi le Zobrist Hashing ?

Nous avons implémenté le hachage de Zobrist, norme pour les IA de jeux de plateau, pour trois raisons :

- **Indépendance de l'historique** : Grâce à l'opérateur binaire XOR ( `^` ), le hachage est commutatif. Que les segments soient tracés dans l'ordre A-B ou B-A, le résultat final est identique.
- **Efficacité algorithmique** : La mise à jour du hash est une opération en  $O(1)$  extrêmement rapide.
- **Minimisation des collisions** : L'utilisation de nombres 64 bits ( `long` ) offre un espace de  $2^{64}$  valeurs possibles.

##### B. Structure et gestion des données (TTEntry)

Chaque entrée stockée dans la table ( `TTEntry` ) contient :

- **La profondeur (depth)** : Une valeur mémorisée n'est réutilisée que si elle a été calculée à une profondeur égale ou supérieure.
- **Les drapeaux de validité (Flags)** : `EXACT` (valeur précise), `LOWER_BOUND` (issue d'une coupure Beta), `UPPER_BOUND` (issue d'une coupure Alpha).
- **Le meilleur coup (bestAction)** : En stockant le meilleur coup trouvé lors d'une recherche précédente, nous pouvons le tester en priorité au prochain passage sur cet état (Move Ordering), augmentant radicalement la probabilité de déclencher des coupures précoces.

##### C. Gestion de la structure de stockage : TranspositionTable

La classe `TranspositionTable` agit comme une mémoire à court terme. Son accès est instantané (complexité en  $O(1)$  via `HashMap` ), mais sa gestion pose un défi : l'explosion combinatoire.

Pour pallier la saturation mémoire, nous avons implémenté une limite stricte de capacité ( `TAILLE_MAX` ). Quand la table est pleine, elle est intégralement vidée. Paradoxalement, ce nettoyage dynamique nous a permis d'augmenter le taux de hits tout au long de la partie, car la

table se remplit immédiatement avec des états frais liés à la branche actuellement explorée. La méthode `clear()` est également appelée à l'initialisation de chaque nouveau match.

### 4.1.3 Optimisations avancées de l'arbre de recherche

**A. L'ordonnement des coups (Move Ordering) :** La méthode `reordonnerAvecTT` interroge la Table de Transposition et place en première position le `bestAction` déjà enregistré, déclenchant des élagages très précoces.

**B. Extension dynamique de l'horizon (Gestion du rejeu) :** Si une action entraîne la fermeture d'une case, la profondeur de recherche n'est pas décrémentée. Cela permet à notre IA d'évaluer l'intégralité d'une chaîne de cases comme un seul coup, atteignant des profondeurs apparentes allant jusqu'à plus de 20.

**C. Intégration d'une Heuristique Experte :** Aux nœuds feuilles, l'algorithme invoque `heuristicv2`, une fonction d'évaluation capable d'analyser les chaînes de cases, d'anticiper la théorie de la parité et d'évaluer la mobilité sans instancier le moindre objet en mémoire.

---

## 4.2 Tests et évaluation des performances : L'infrastructure AIArena

Pour sortir du cadre des tests manuels, nous avons développé la classe `AIArena`. Cet outil simule des confrontations automatisées en reproduisant fidèlement les contraintes du tournoi final, avec des matchs aller-retour (inversion des rôles pour compenser l'avantage du premier/deuxième joueur) et un respect strict de l'horloge.

`AIArena` nous a également servi d'outil de diagnostic, permettant d'analyser la profondeur d'exploration et le temps de calcul par nœud. Cette analyse nous a conduits à implémenter une **répartition dynamique du temps** basée sur le nombre de segments restants :

$$\text{temps\_alloué} = \text{temps\_restant} / (\text{segments\_restants} / K)$$

En ajustant la constante  $K$ , l'IA « prend son temps » dans les ouvertures complexes puis accélère en fin de partie lorsque la table de transposition lui permet de résoudre le plateau quasi instantanément.

---

## 4.3 Amélioration de l'heuristique

### 4.3.1 Architecture et concepts tactiques de l'Heuristique v2

L'intégration de la Table de Transposition a mis en lumière le manque de performance de notre fonction d'évaluation originelle. Nous avons conçu `heuristicv2` dont la philosophie repose sur la **performance sans allocation mémoire** et l'**analyse structurale du plateau**.

**1. Évaluation matricielle bas niveau (Le coût de la mobilité) :** L'heuristique scanne directement les matrices booléennes des arêtes horizontales et verticales, avec une pénalité asymétrique : fermer 1 ou 2 cases donne un bonus positif massif, tandis que donner une case inflige une lourde pénalité.

**2. Analyse des chaînes via Union-Find (Anticipation de fin de partie) :** Pour détecter les chaînes sans ralentir le calcul, nous avons implémenté une structure Union-Find. Elle regroupe les cases adjacentes ( $\geq 2$  côtés tracés) en composantes connexes via de simples tableaux d'entiers (`parent`, `rank`, `compSize`), et distingue les chaînes (avec une entrée à 3 côtés) des boucles (cycles fermés à 2 côtés).

**3. Stratégie avancée : Parité et Double-Dealing :** En calculant la parité des coups sûrs restants et le nombre de chaînes longues, l'heuristique détermine quel joueur sera forcé d'ouvrir la première chaîne (ouverture forcée). Elle simule également automatiquement la stratégie experte du *double-dealing* : l'algorithme calcule un gain net qui assume que l'IA va sacrifier volontairement les 2 dernières cases d'une chaîne pour forcer l'adversaire à ouvrir la chaîne suivante.

**4. Poids dynamiques selon la phase de jeu :** L'heuristique calcule un ratio représentant la phase de jeu (arêtes jouées / total). La mobilité est fortement valorisée en début de partie, tandis que le contrôle des chaînes et les pénalités de dangerosité deviennent les critères dominants en fin de partie.

---

## 4.4 Approche alternative : Monte-Carlo Tree Search (MCTS)

### 4.4.1 Principe de l'algorithme

MCTS repose sur quatre phases répétées en boucle :

1. **Sélection** : Depuis la racine, descendre dans l'arbre en choisissant à chaque nœud l'enfant qui maximise la formule UCT :  $UCT = (\text{wins} / \text{visits}) + C \times \sqrt{(\ln(\text{parent\_visits}) / \text{visits})}$  où  $C = 1.0$ .
2. **Expansion** : Lorsqu'un nœud possède encore des actions non explorées, en sélectionner une et créer un nœud enfant.
3. **Simulation (rollout)** : Simuler une partie aléatoire semi-informée jusqu'à la fin.
4. **Rétropropagation** : Remonter le résultat (victoire/défaite/nul) le long du chemin jusqu'à la racine.

#### 4.4.2 Optimisations implémentées

**Parallélisation multi-thread** : Un pool de threads dimensionné au nombre de cœurs disponibles. Chaque thread construit son propre arbre MCTS indépendamment et les résultats sont fusionnés par agrégation (*root parallelization*).

**Simulations semi-informées** : Trois niveaux de priorité : (1) fermer une case ( `isActionClosing` ), (2) jouer un coup sûr aléatoire, (3) coup totalement aléatoire en dernier recours.

**Détection de victoire anticipée** : Si un joueur possède plus de la moitié des cases totales pendant un rollout, la simulation s'arrête immédiatement.

**Biais heuristique (Prior Knowledge)** : Lors de l'expansion, le score heuristique est converti en probabilité via une sigmoïde et injecté sous forme de 10 « visites fantômes » biaisant l'exploration UCT.

**Technique swap-and-pop** : Suppression en  $O(1)$  d'une action de la liste pendant les rollouts.

#### 4.4.3 Limites observées

Malgré ces optimisations, l'agent MCTS reste **inférieur à Alpha-Beta + TT** sur toutes les tailles de grille. Cela s'explique par la nature très tactique de la Pipopipette : les chaînes de cases créent des effets de cascade déterministes qu'Alpha-Beta capture exactement, tandis que MCTS les évalue de manière probabiliste avec une variance élevée.

---

### 4.5 Pondering : calcul pendant le tour adverse

Pour maximiser le temps de calcul effectif, nous avons implémenté dans `ExpertActionStrategy` un mécanisme de **pondering** :

1. **Après avoir retourné son coup**, l'agent Expert clone le plateau, applique son propre coup, et lance un thread démon ( `ponderThread` ) qui explore l'arbre depuis l'état résultant.
2. **Pendant que l'adversaire réfléchit**, le thread effectue un Iterative Deepening Alpha-Beta, alimentant la Table de Transposition.
3. **Quand c'est à nouveau le tour de l'agent**, le thread est interrompu et l'agent dispose d'une table de transposition pré-remplie.

```
Board clonedBoard = new Board(board);
int nbFerme = clonedBoard.apply(meilleurCoupTrouve, playerId);
int nextPlayerId = (nbFerme > 0) ? playerId : 1 - playerId;

ponderThread = new Thread(() -> {
    while (depth <= actionsSize && !Thread.currentThread().isInterrupted()) {
        alphaBetaTT(clonedBoard, depth, nextPlayerId, ...);
        depth++;
    }
});
ponderThread.setDaemon(true);
ponderThread.start();
```

**Impact observé** : Particulièrement efficace lorsque l'adversaire joue le coup anticipé — le temps de décision initial est réduit à souvent <50 ms au lieu de >500 ms.

---

### 4.6 Résultats du tournoi interne

Tournoi complet via `AIArena` sur les grilles 3×3, 4×4, 4×5, 5×5, 6×6 et 6×8 (matches aller-retour). Taux de victoire de l'agent A (lignes) contre l'agent B (colonnes) :

| Agent A ↓ \ Agent B → | Random | Glouton | Alpha-Beta (sans TT) | Alpha-Beta + TT | Expert (TT + Pondering) | MCTS |
|-----------------------|--------|---------|----------------------|-----------------|-------------------------|------|
| Glouton               | ~90%   | —       | —                    | —               | —                       | —    |
| Alpha-Beta (sans TT)  | ~100%  | ~95%    | —                    | —               | —                       | —    |
| Alpha-Beta + TT       | ~100%  | ~100%   | ~80%                 | —               | —                       | —    |
| Expert                | ~100%  | ~100%   | ~85%                 | ~55%            | —                       | ~90% |
| MCTS                  | ~100%  | ~95%    | ~50%                 | ~20%            | ~10%                    | —    |

#### Gain mesuré par rapport à Alpha-Beta seul :

- Sur grilles 3×3 et 4×4 : gain marginal (~55-60% de victoires).
- Sur grilles 5×5 et 6×6 : gain significatif (~80% de victoires).
- Sur grille 6×8 : avantage très net (~90% de victoires).

Ces résultats démontrent un **gain mesurable et significatif** de l'agent Expert par rapport à Minimax et Alpha-Beta seuls, principalement grâce à la Table de Transposition, l'heuristique v2, et le Pondering.